

CoreFly F2

飞控软件设计计划书

版本: v2.0 (详尽版) | 日期: 2026-05-23

适用硬件: STM32F407VET6 + FD6288Q + AON7418 × 24

适用电机: 4路 BLDC (无感 FOC 驱动)

—— 逐行对着写就能实现的开发手册 ——

目录

第一部分: 硬件平台与现状回顾

- 一、硬件平台概述
- 二、项目当前代码进度

第二部分: FOC 理论基础 (必读)

- 三、FOC 完整数学推导
 - 3.1 Clarke 变换
 - 3.2 Park 变换
 - 3.3 逆 Park 变换
 - 3.4 SVPWM (空间矢量 PWM) 完整推导
- 四、无感 FOC 角度估算理论
 - 4.1 反电动势与转子角度
 - 4.2 滑模观测器 (SMO) 完整推导
 - 4.3 锁相环 (PLL) 速度提取

第三部分: 分阶段开发详细方案

- 第0阶段: 开发环境与工具链确认
- 第1阶段: 单路电机 FOC 完整实现
 - 1.1 PWM 输出验证
 - 1.2 ADC 电流采样
 - 1.3 通用 PID 控制器
 - 1.4 SMO + PLL 观测器
 - 1.5 电流环闭环
 - 1.6 电机启动流程 (预定位→开环强拖→切入闭环)
- 第2阶段: 传感器与姿态解算
- 第3阶段: 四电机扩展与飞控核心
- 第4阶段: 通信与安全监控
- 第5阶段: 系统集成与飞行测试

第一部分: 硬件平台与现状回顾

- 一、硬件平台概述
 - 1.1 主控芯片
 - 1.2 电机驱动拓扑 (单路)
 - 1.3 PWM 定时器分配 (4路电机)
 - 1.4 PWM 参数
 - 1.5 ADC 电流采样
 - 1.6 传感器
 - 1.7 引脚分配速查

二、项目当前代码进度

第二部分: FOC 理论基础 (必读)

三、FOC 完整数学推导

- 3.1 Clarke 变换 ($abc \rightarrow \alpha-\beta$)
- 3.2 Park 变换 ($\alpha-\beta \rightarrow d-q$)
- 3.3 逆 Park 变换 ($d-q \rightarrow \alpha-\beta$)
- 3.4 SVPWM 完整推导 (空间矢量脉宽调制)

四、无感 FOC 角度估算理论

- 4.1 为什么需要角度估算?
- 4.2 电机在 $\alpha-\beta$ 坐标系下的电压方程
- 4.3 反电动势与转子角度的关系
- 4.4 滑模观测器 (SMO) 完整推导
- 4.5 锁相环 (PLL) 速度提取

第三部分: 分阶段开发详细方案

第0阶段: 开发环境与工具链确认 (1天)

第1阶段: 单路电机 FOC 完整实现 (核心中的核心, 预计 10-14 天)

- 1.1 PWM 输出验证 (2天)
- 1.2 ADC 电流采样 (3天)
- 1.3 通用 PID 控制器 (2天)
- 1.4 SMO + PLL 观测器实现 (4天)
- 1.5 电流环闭环 (3天)
- 1.6 电机启动流程 (3天)

第2阶段: 传感器与姿态解算 (预计 5-7 天)

- 2.1 补全 SPL06-001 气压计驱动 (1天)
- 2.2 姿态解算 Mahony 互补滤波 (3天)
- 2.3 传感器融合任务 App_Sensor (1天)

第3阶段: 四电机扩展与飞控核心 (预计 7-10 天)

- 3.1 将 App_Motor_Ctrl 扩展为 4 电机 (2天)
- 3.2 姿态控制任务 (3天)
- 3.3 地面测试 (2天)

第4阶段: 通信与安全监控 (预计 4-6 天)

- 4.1 LoRa 驱动 (2天)
- 4.2 遥控数据解析 (2天)
- 4.3 状态监控任务 (1天)

第5阶段: 系统集成与飞行测试 (预计 5-7 天)

- 5.1 系留飞行测试
- 5.2 PID 整定 (飞行中)
- 5.3 自由飞行测试

附录A: PID 参数整定方法 (经验法)

附录B: 电机参数测量方法 (动手篇)

附录C: 调试命令参考 (通过 VOFA+ 发送)

附录D: 常见故障排查表

=====

=====
编写日期: 2026-05-23
适用硬件: STM32F407VET6 + FD6288Q + AON7418 × 24
适用电机: 4路 BLDC (无感 FOC 驱动)

说明:
本文档不是概念性规划, 而是一份"逐行对着写就能实现"的开发手册。
每个阶段都包含了: 理论基础 → 数学公式 → 代码实现要点 → 验证方法 → 常见坑点。

=====

=====
=====

=====

第一部分：硬件平台与现状回顾

- 一、硬件平台概述
- 二、项目当前代码进度

第二部分：FOC 理论基础（必读）

三、FOC 完整数学推导

- 3.1 Clarke 变换
- 3.2 Park 变换
- 3.3 逆 Park 变换
- 3.4 SVPWM (空间矢量 PWM) 完整推导

四、无感 FOC 角度估算理论

- 4.1 反电动势与转子角度
- 4.2 滑模观测器 (SMO) 完整推导
- 4.3 锁相环 (PLL) 速度提取

第三部分：分阶段开发详细方案

第0阶段: 开发环境与工具链确认

第1阶段: 单路电机 FOC 完整实现

1.1 PWM 输出验证

1.2 ADC 电流采样

1.3 通用 PID 控制器

1.4 SMO + PLL 观测器

1.5 电流环闭环

1.6 电机启动流程 (预定位→开环强拖→切入闭环)

第2阶段: 传感器与姿态解算

第3阶段: 四电机扩展与飞控核心

第4阶段: 通信与安全监控

第5阶段: 系统集成与飞行测试

附录

A. PID 参数整定方法

B. 电机参数测量方法

C. 调试命令参考

D. 常见故障排查表

=====

=====

第一部分：硬件平台与现状回顾

=====

=====

一、硬件平台概述

1.1 主控芯片

MCU: STM32F407VET6

内核: ARM Cortex-M4F (带硬件 FPU + DSP 指令)

主频: 168MHz

Flash: 512KB

SRAM: 192KB

封装: LQFP-100

IDE: Keil MDK-ARM V5.32

RTOS: FreeRTOS (tick 1000Hz, heap_4, 30KB 堆)

1.2 电机驱动拓扑 (单路)

```
| STM32F407 高级定时器 TIMx |  
| → CH1/CH1N, CH2/CH2N, CH3/CH3N (6路互补PWM) |  
| → FD6288Q 三相栅极驱动器 |  
| └─ 内置死区时间控制 |  
| └─ 自举二极管 (Bootstrap) |  
| └─ 故障检测 (FAULT 引脚) |  
| → 6× AON7418 N-MOSFET 三相全桥 |  
| → BLDC 三相绕组 (U/V/W) |  
| → 双电阻电流采样 (U相下桥臂 + V相下桥臂) |
```

1.3 PWM 定时器分配 (4路电机)

电机1 (TIM1, 高级): CH1/CH1N → M1_UH/UL, CH2/CH2N → M1_VH/VL, CH3/CH3N → M1_WH/WL

电机2 (TIM8, 高级): CH1/CH1N → M2_UH/UL, CH2/CH2N → M2_VH/VL, CH3/CH3N → M2_WH/WL

电机3 (TIM2+TIM9): TIM2 CH1/2/3 → M3_UH/VH/WH, TIM9 CH1/2 → M3_VL/WL, TIM2 CH4 → M3_UL

电机4 (TIM4+TIM10/11): TIM4 CH1/2/3/4 → M4_UH/VH/WH/UL, TIM10 → M4_VL, TIM11 → M4_WL

注意: 电机3/4 使用通用定时器拼凑, 无硬件互补输出和死区。

初步调试阶段优先使用 TIM1 (电机1) 和 TIM8 (电机2)。

1.4 PWM 参数

频率: 20kHz (人耳听不到, 开关损耗可接受)

计数模式: 中心对齐 (Center-Aligned) 模式1

TIM_CLK: 168MHz

PSC: 0 (不分频)

ARR: 8400 ($168\text{MHz} / 8400 / 2 = 10\text{kHz} \rightarrow$ 中心对齐等效 20kHz)

占空比换算: $\text{duty} = \text{CCR} / \text{ARR}$

50% 占空比: $\text{CCR} = 4200$

0% 占空比: $\text{CCR} = 0$

100% 占空比: $\text{CCR} = 8400$

1.5 ADC 电流采样

采样方案: 双电阻采样 (U 相 + V 相下桥臂各串采样电阻)
放大芯片: TPA1832 / TPA1812 运放
ADC 外设: ADC1 (电机1/2 电流), ADC2 (电机3/4 电流 + VBAT)
触发方式: TIM1_TRGO (在 PWM 中心点触发, 避开开关噪声)
采样模式: 注入组 (Injected Group), 优先级高于规则组
搬运方式: DMA (无需 CPU 干预)

1.6 传感器

IMU: BMI160 (SPI1, 加速度计+陀螺仪, ODR=1600Hz)
气压计: SPL06-001 (I2C1, 地址 0x76)
通信: Ra-01SC / SX1278 (SPI3, LoRa)
调试串口: USART1 (VOFA+ Firewater 协议, 115200bps)
接收机: USART3 (遥控信号输入)

1.7 引脚分配速查

LED_RED: PD2
LED_BLUE: PD3
BEEP: PE0
IMU_NSS: PA4
IMU_INT: PD1 (EXTI1)
LORA_NSS: PD6
LORA_DIO0: PD7 (EXTI9_5)
LORA_Nrst: PD4
LORA_TXEN: PD5
LORA_RXEN: PD10
I2C1_SCL: PB6
I2C1_SDA: PB7
USART3_TX: PD8
USART3_RX: PD9

二、项目当前代码进度

[已完成] Alg_FOC_Math.c/h — Clarke / Park / 逆Park / SVPWM 完整实现
[已完成] App_Motor_Ctrl.c/h — 单电机控制器 (开环强拖已跑通)
[已完成] App_FreeRTOS.c/h — FreeRTOS 任务框架 (Zhuanhuan_task @ 500Hz)
[已完成] BSP_BMI160.c/h — BMI160 驱动 (寄存器定义/初始化/读取)
[已完成] BSP_USART.c/h — printf 重定向到 VOFA+
[已完成] HAL 层 — GPIO / SPI / I2C / TIM1/2/4/8/9/10/11/12 / ADC
[仅骨架] BSP_SPL06.c/h — 寄存器定义完整, 功能未实现
[空文件] BSP_Ra-01SC.c/h — LoRa 驱动未实现
[未开始] Alg_PID — PID 控制器
[未开始] Alg_Attitude — 姿态解算 (Mahony)
[未开始] SMO 观测器 — 无感 FOC 角度估算

[未开始] 多电机支持 — 当前仅 Motor_Ctrl_t MyMotor (单电机)

=====

=====

第二部分：FOC 理论基础（必读）

=====

=====

在写代码之前，必须理解下面这些公式的含义和物理意义。

FOC 的核心思想：把交流电机等效为直流电机来控制。

BLDC 的三相电流 (I_a , I_b , I_c) 是正弦波，在定子中产生旋转磁场。

转子 (永磁体) 跟随旋转磁场转动。

如果我们在"转子的视角"来看电流，正弦波就变成了直流量。

这就是坐标变换的意义：

三相静止 (abc) $\rightarrow$ 两相静止 (α - β) $\rightarrow$ 两相旋转 (d-q)

在 d-q 坐标系下：

I_d (d轴电流) = 励磁电流，对于表贴式 PMSM，永远希望 $I_d = 0$

I_q (q轴电流) = 转矩电流，正比于电机输出扭矩

所以我们只需要控制 I_q ，就像控制直流电机的电枢电流一样。

三、FOC 完整数学推导

3.1 Clarke 变换 ($abc \rightarrow \alpha\beta$)

目的: 将 120° 间隔的三相电流变换到正交的 α - β 坐标系。

矩阵形式:

$$\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = \frac{2}{3} \begin{bmatrix} 1 & -1/2 & -1/2 \\ 0 & \sqrt{3}/2 & -\sqrt{3}/2 \end{bmatrix} \begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix}$$

利用基尔霍夫电流定律 $I_a + I_b + I_c = 0 \rightarrow I_c = -(I_a + I_b)$ ，可以化简为:

$$\begin{aligned} I_\alpha &= I_a \\ I_\beta &= (I_a + 2 \times I_b) / \sqrt{3} \\ &= (I_a + 2 \times I_b) \times 0.57735027 \end{aligned}$$

实现要点:

- 预定义 $\text{FOC_INV_SQRT3} = 0.57735027f$, 把除以 $\sqrt{3}$ 变成乘法
- 只需要采样两相电流 (I_a, I_b), I_c 由 KCL 算出

当前代码位置: Alg_FOC_Math.c $\rightarrow$ Alg_FOC_Clarke()

3.2 Park 变换 ($\alpha\text{-}\beta \rightarrow d\text{-}q$)

目的: 将静止的 $\alpha\text{-}\beta$ 坐标系变换到跟随转子旋转的 $d\text{-}q$ 坐标系。

物理图像:

- $\alpha\text{-}\beta$ 是定子视角: 电流是正弦波, 不直观
- $d\text{-}q$ 是转子视角: 电流变成直流量, 可以用简单的 PI 控制

$$\begin{aligned} & \text{公式} \\ & \begin{cases} I_d = I_\alpha \cos\theta + I_\beta \sin\theta \\ I_q = -I_\alpha \sin\theta + I_\beta \cos\theta \end{cases} \\ & \text{其中 } \theta = \text{转子电角度 (弧度)} \end{aligned}$$

矩阵形式 (旋转矩阵):

$$\begin{bmatrix} I_d \\ I_q \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix}$$

当前代码位置: Alg_FOC_Math.c $\rightarrow$ Alg_FOC_Park()

3.3 逆 Park 变换 ($d\text{-}q \rightarrow \alpha\text{-}\beta$)

目的: 把 PID 算出的目标电压 ($V_d\text{-target}, V_q\text{-target}$) 从旋转坐标系变回静止坐标系, 为生成 PWM 做准备。

$$\begin{aligned} & \text{公式} \\ & \begin{cases} V_\alpha = V_d \cos\theta - V_q \sin\theta \\ V_\beta = V_d \sin\theta + V_q \cos\theta \end{cases} \end{aligned}$$

矩阵形式 (逆转矩阵):

$$\begin{bmatrix} V_\alpha \\ V_\beta \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} V_d \\ V_q \end{bmatrix}$$

当前代码位置: Alg_FOC_Math.c $\rightarrow$ Alg_FOC_InvPark()

3.4 SVPWM 完整推导 (空间矢量脉宽调制)

3.4.1 为什么要用 SVPWM?

最简单的做法是 SPWM (正弦 PWM):

直接将 V_α, V_β 做逆 Clarke 得到三相正弦波, 映射到 PWM 占空比。

但 SPWM 的母线电压利用率只有 86.6%。

SVPWM 通过注入零序分量 (共模电压), 把电压利用率提升到 100%。
波形从正弦波变成"马鞍波", 峰值削平, 基波幅值可以更大。

3.4.2 8 个基本电压矢量

三相半桥有 8 种开关状态 (每个半桥上管开=1, 下管开=0):

状态 (a,b,c) | 矢量名 | 说明

000	V0	零矢量
001	V5	C 相上管开
010	V3	B 相上管开
011	V4	B,C 相上管开
100	V1	A 相上管开
101	V6	A,C 相上管开
110	V2	A,B 相上管开
111	V7	零矢量

这 6 个非零矢量在 α - β 平面上将空间分成 6 个扇区, 每扇区 60°。

3.4.3 SVPWM 简化实现 (零序注入法)

实际代码中不需要做扇区判断和矢量时间计算。

使用"零序分量注入法"可以一步到位:

```
| |
| 步骤1: 逆 Clarke 变换 ( $\alpha$ - $\beta \rightarrow$  三相虚拟电压) |
| |
|  $v_a = V_\alpha$  |
|  $v_b = (-V_\alpha + \sqrt{3} \times V_\beta) / 2$  |
|  $v_c = (-V_\alpha - \sqrt{3} \times V_\beta) / 2$  |
| |
| 步骤2: 计算零序分量 (共模偏置电压) |
| |
|  $v\_offset = (\max(v_a, v_b, v_c) + \min(v_a, v_b, v_c)) / 2$  |
| |
| 步骤3: 注入零序分量, 得到最终相电压 |
| |
|  $U_a = v_a - v\_offset$  |
|  $U_b = v_b - v\_offset$  |
|  $U_c = v_c - v\_offset$  |
| |
| 步骤4: 归一化到 PWM 占空比 (0~1) |
| |
|  $duty\_x = U_x / V_{Bus} + 0.5$  |
| |
| 步骤5: 映射到定时器 CCR 值 |
| |
|  $CCR\_x = (\text{uint32\_t})(duty\_x \times ARR)$  |
| (限制在 [0, ARR] 范围内, 防止炸管) |
| |
```

3.4.4 为什么加上 0.5?

在中心对齐 PWM 模式下, $CCR = ARR/2$ 代表 50% 占空比。
经过 SVPWM 后 U_a 的范围大约是 $[-V_{Bus}/2, +V_{Bus}/2]$ 。
加 0.5 后占空比范围变成 $[0.25, 0.75]$ 左右 (取决于 V_{q_target} 大小)。
也就是说: 50% 占空比 = 电机相电压为 0 (电机不转)。
>50% = 正电压, <50% = 负电压。
当前代码位置: Alg_FOC_Math.c → Alg_FOC_SVPWM()

3.4.5 SVPWM 的马鞍波验证

用 VOFA+ 看 CCR_a 的波形:
- SPWM: 纯正弦波
- SVPWM: 正弦波的波峰被"压扁" → 马鞍波
如果你看到的是马鞍波, 说明 SVPWM 实现正确。

四、无感 FOC 角度估算理论

4.1 为什么需要角度估算?

没有霍尔传感器或编码器, 我们不知道转子在哪个位置。
但我们可以通过电机的电压和电流反推转子位置。
核心原理: 永磁体转子旋转时, 会在定子绕组中产生反电动势 (BEMF)。
BEMF 的相位直接对应转子角度。

4.2 电机在 α - β 坐标系下的电压方程

$$\begin{aligned} V_\alpha &= R \cdot I_\alpha + L \cdot \frac{dI_\alpha}{dt} + E_\alpha \\ V_\beta &= R \cdot I_\beta + L \cdot \frac{dI_\beta}{dt} + E_\beta \end{aligned}$$

其中:

V_α, V_β = 施加的定子电压 (已知, 来自逆Park)

I_α, I_β = 定子电流 (已知, 来自 Clarke)

R = 相电阻 (需要实测)

L = 相电感 (需要实测)

E_α, E_β = 反电动势 (未知, 需要估算)

4.3 反电动势与转子角度的关系

$$\begin{aligned} E_\alpha &= -K_e \times \omega_e \times \sin(\theta_e) \\ E_\beta &= K_e \times \omega_e \times \cos(\theta_e) \end{aligned}$$

其中:

K_e = 反电动势常数 (V·s/rad)

ω_e = 电角速度 (rad/s)

```

|  $\theta_e$  = 电角度 (rad) |
| |
| 一旦估算出  $E_\alpha$  和  $E_\beta$ , 角度就可以通过反正切得到: |
| |
|  $\theta_e = \text{atan2}(-E_\alpha, E_\beta)$  |
| |
|_____|

```

4.4 滑模观测器 (SMO) 完整推导

4.4.1 为什么用 SMO 而不是直接算?

直接算 $E_\alpha = V_\alpha - R \cdot I_\alpha - L \cdot dI_\alpha/dt$ 需要对电流求导,
 实际系统中电流有大量噪声, 微分操作会放大噪声, 导致角度估算失败。

SMO 的思路: 不直接算 BEMF, 而是构造一个"虚拟电机模型",
 通过比较实际电流和虚拟模型电流的误差, 用滑模控制将误差强制归零,
 此时虚拟模型的 BEMF 就是真实 BEMF。

4.4.2 SMO 电流观测器方程

```

| |
| 构造电流观测器 (估算  $\alpha$ - $\beta$  电流): |
| |
|  $d(I\hat{I}_\alpha)/dt = (-R/L) \cdot I\hat{I}_\alpha + (1/L) \cdot V_\alpha - (K/L) \cdot \text{sat}(I\hat{I}_\alpha - I_\alpha)$  |
|  $d(I\hat{I}_\beta)/dt = (-R/L) \cdot I\hat{I}_\beta + (1/L) \cdot V_\beta - (K/L) \cdot \text{sat}(I\hat{I}_\beta - I_\beta)$  |
| |
| 其中: |
|  $I\hat{I}_\alpha, I\hat{I}_\beta$  = 估算电流 |
|  $I_\alpha, I_\beta$  = 实际采样电流 |
|  $K$  = 滑模增益 (需整定, 通常取  $K > \max(|E|)$ ) |
|  $\text{sat}(x)$  = 饱和函数 (代替 sign 函数减少抖振) |
| |
|_____|

```

4.4.3 饱和函数 $\text{sat}(x)$

使用饱和函数代替符号函数 $\text{sign}(x)$, 可以大幅减少滑模抖振:

```

| |
|  $\begin{cases} 1 & \text{if } x > \delta \\ \text{sat}(x) = x/\delta & \text{if } |x| \leq \delta \\ -1 & \text{if } x < -\delta \end{cases}$  |
| |
| 其中  $\delta$  = 边界层厚度 (典型值: 0.5 ~ 2.0) |
| |
|_____|

```

4.4.4 BEMF 提取

当滑模面到达 ($I\hat{I} \approx I$) 时, 切换函数 $\text{sat}(I\hat{I} - I)$ 的平均值就是 BEMF:

```

| |
|  $E\hat{I}_\alpha = K \times \text{sat}(I\hat{I}_\alpha - I_\alpha)$  (需要低通滤波!) |
|  $E\hat{I}_\beta = K \times \text{sat}(I\hat{I}_\beta - I_\beta)$  (需要低通滤波!) |
| |
|_____|

```

```
| 一阶低通滤波器: |
|  $E_{\alpha\_filtered}[n] = E_{\alpha\_filtered}[n-1] + (T_s/\tau) \cdot (E_{\alpha\_raw} - E_{\alpha\_filtered}[n-1])$  |
| |
| 其中  $T_s$  = 控制周期,  $\tau$  = 滤波器时间常数 (~0.001s) |
| |
```

4.4.5 离散化实现 (可以直接写成 C 代码)

实际的离散化 SMO 方程 (使用前向欧拉法, T_s 为控制周期):

```
| |
| // 1. 电流误差 |
|  $err_{\alpha} = I_{\hat{\alpha}} - I_{\alpha}$  |
|  $err_{\beta} = I_{\hat{\beta}} - I_{\beta}$  |
| |
| // 2. 滑模控制量 (饱和函数) |
|  $z_{\alpha} = K_{slide} \times \text{sat}(err_{\alpha}, \text{delta})$  |
|  $z_{\beta} = K_{slide} \times \text{sat}(err_{\beta}, \text{delta})$  |
| |
| // 3. 更新电流估算 (前向欧拉) |
|  $I_{\hat{\alpha}} += T_s \times [(-R/L) \times I_{\hat{\alpha}} + (1/L) \times V_{\alpha} - (1/L) \times z_{\alpha}]$  |
|  $I_{\hat{\beta}} += T_s \times [(-R/L) \times I_{\hat{\beta}} + (1/L) \times V_{\beta} - (1/L) \times z_{\beta}]$  |
| |
| // 4. 低通滤波提取 BEMF |
|  $E_{\alpha} += T_s \times (1/\tau) \times (z_{\alpha} - E_{\alpha})$  //  $\tau \approx 0.5\text{ms} \sim 2\text{ms}$  |
|  $E_{\beta} += T_s \times (1/\tau) \times (z_{\beta} - E_{\beta})$  |
| |
| // 5. 反正切求角度 |
|  $\theta_{est} = \text{atan2f}(-E_{\alpha}, E_{\beta})$  |
| |
| // 6. 角度补偿 (低通滤波器引入的相位滞后) |
|  $\theta_{comp} = \theta_{est} + \omega_e \times \tau_{lpf}$  |
| |
```

4.4.6 SMO 参数整定指南

参数	如何整定
K_{slide}	初始值 = $3 \times \max(\text{反电动势峰值})$ 反电动势峰值 $\approx K_e \times \omega_{rated}$ 太小: 收敛慢; 太大: 抖振严重
delta	初始值 = 1.0 太小: 接近 sign 函数, 抖振大 太大: 滑模特性退化, 估算精度下降
τ (LPF)	初始值 = 0.001 (1ms) 太小: BEMF 噪声大, 角度抖 太大: 相位滞后严重, 高速时角度误差大

4.5 锁相环 (PLL) 速度提取

直接从 SMO 估算的角度通过差分求速度会放大噪声。

PLL 利用 BEMF 的相位信息，通过闭环方式平滑地跟踪角度和速度。

```
| |
| PLL 输入: E_α, E_β (SMO 估算的 BEMF) |
| |
| // 1. 相位误差 (通过 BEMF 矢量点积计算) |
| err = -E_α × cos(θ_pll) - E_β × sin(θ_pll) |
| |
| // 2. PI 控制器 (将相位误差调节为 0) |
| integral += err × dt |
| ω_pll = Kp_pll × err + Ki_pll × integral |
| |
| // 3. 积分得到角度 |
| θ_pll += ω_pll × dt |
| |
| // 4. 角度归一化 |
| if(θ_pll > PI) θ_pll -= 2×PI |
| if(θ_pll < -PI) θ_pll += 2×PI |
| |
| PLL 带宽 (典型值): 100 ~ 500 rad/s |
| Kp_pll = 2 × BW |
| Ki_pll = BW² |
| |
```

PLL vs 反正切:

- 反正切 atan2: 响应快，但噪声大，需要滤波
- PLL: 响应稍慢，但输出平滑，同时给出角度和速度
- 推荐: 两种都实现，反正切用于快速检测，PLL 用于稳态运行

=====

=====

第三部分：分阶段开发详细方案

=====

=====

核心原则: 每一步都要有可验证的输出。不写"半成品"代码堆积。

第0阶段: 开发环境与工具链确认 (1天)

目标: 确保编译、烧录、调试、波形查看全链路通畅。

任务0.1: 确认编译环境

- 打开 Keil MDK-ARM，编译项目 CoreFly F2，0 错误 0 警告

- 确认优化等级: -O2 (不要用 -O0, FOC 数学计算量大)
- 确认 FPU 已启用: 工程选项 → Target → Floating Point Hardware → "Use FPU"

任务0.2: 确认烧录与调试

- 连接 ST-Link / J-Link, 烧录固件
- 确认 LED 闪烁 (证明程序在跑)
- 确认串口 VOFA+ 能收到 printf 数据

任务0.3: 确认 VOFA+ 配置

- 打开 VOFA+ → 协议选择 "Firewater"
- 波特率 115200
- 配置 3 个波形通道: CCR_a, CCR_b, CCR_c (对应 printf 的前 3 个数据)
- 确认能看到当前 Zhuanhuan_task 输出的马鞍波

任务0.4: 配置 Keil Watch 窗口

- 添加 MyMotor 结构体到 Watch 窗口
- 实时观察 Theta, V_q_target, I_alpha, I_beta 等变量

验证标准:

- Keil 编译 0 错误 0 警告
- ST-Link 烧录成功
- VOFA+ 收到 printf 数据: 3路 CCR 波形
- VOFA+ 波形呈现马鞍波 (确认 SVPWM 正确)

第1阶段: 单路电机 FOC 完整实现 (核心中的核心, 预计 10-14 天)

目标: 让一个电机动起来 — 从开环强拖到电流闭环稳定运行。

这也是整个项目中最难、最重要的阶段。

□ 重要提醒: 电机1 使用 TIM1 (PA8/PA9/PA10 为 HIN, PB0/PB1/PE8 为 LIN)。

确保 FD6288Q 已焊接、MOSFET 已焊接、采样电阻已焊接、电机已连接。

第1阶段子任务总览
1.1 PWM 输出验证 → 示波器看6路波形
1.2 ADC 电流采样 → VOFA+ 看三相电流
1.3 通用 PID 库 → 单元测试验证
1.4 SMO+PLL观测器 → VOFA+ 看估算角度跟踪
1.5 电流环闭环 → VOFA+ 看 Id/Iq 跟踪
1.6 启动流程 → 预定位→强拖→闭环全流程

1.1 PWM 输出验证 (2天)

1.1.1 确认 CubeMX 定时器配置

打开 CoreFly F2.ioc, 检查 TIM1 配置:

```

| TIM1 模式与配置 |
|
| Clock Source: Internal Clock |
| Channel1: PWM Generation CH1 CH1N |
| Channel2: PWM Generation CH2 CH2N |
| Channel3: PWM Generation CH3 CH3N |
|
| Counter Mode: Center Aligned mode 1 |
| (CMS[1:0]=01, DIR=11 即从0向上计数再到0) |
|
| Prescaler: 0 (PSC = 0, 不分频) |
| Counter Period: 8400 (ARR) |
|
| PWM频率 = 168MHz / (2 × 8400) = 10kHz |
| 等效PWM频率 (中心对齐) = 20kHz |
|
| Auto-reload preload: Enable (ARPE=1) |
|
| Dead Time: ~100ns (DTG 参考 FD6288Q datasheet) |
| Break (BKIN): Disable (暂无需要) |
|
| CH Polarity: High (正常极性, CCR越大占空比越大) |
| CHN Polarity: High |
| CH Idle State: Reset (空闲时为低电平, MOSFET关断) |
|
|_____

```

1.1.2 手动 PWM 输出测试 (不接电机!)

□ 安全警告: 先不接电机! 先用示波器看波形!

写一个最简单的测试: 直接在 main.c 或测试任务中手动设置 CCR 值。

```

// ===== 测试代码: 50% 占空比输出 =====
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_1); // 互补通道!
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
HAL_TIMEx_PWMN_Start(&htim1, TIM_CHANNEL_3);
// 50% 占空比 → 三相电压相等 → 电机不转
TIM1->CCR1 = 4200; // A 相
TIM1->CCR2 = 4200; // B 相
TIM1->CCR3 = 4200; // C 相

```

验证项目:

- 示波器 CH1 测 TIM1_CH1 (PA8): 应该是 20kHz 方波, 占空比 50%
- 示波器 CH2 测 TIM1_CH1N (PB13): 应该是 20kHz 方波, 占空比 50%
- CH1 和 CH1N 之间应该有一个很短的死区时间 (~100ns)
- 两路波形互为互补 (一个高电平时另一个低电平)
- 确认死区: 两路不会同时为高 → 否则上下管直通短路!

常见问题排查:

- 看不到互补输出? → 确认 HAL_TIMEx_PWMN_Start() 已调用
- 占空比不对? → 确认计数模式和 ARR 值
- 没有死区? → 检查 CubeMX → TIM1 → Break and Dead-Time settings
- 波形抖动? → 确认 PLL 时钟配置正确 (168MHz)

1.1.3 开环 SVPWM 输出测试 (接电机, 不装桨!)

使用现有的 Zhuanhuan_task 代码验证:

```
| 当前代码逻辑回顾 (App_FreeRTOS.c: Zhuanhuan_task): |  
| |  
| App_Motor_Init(&MyMotor, 12.0f, 8400); |  
| MyMotor.MathParam.V_q_target = 4.0f; // Q轴电压 |  
| App_Motor_Set_Speed(&MyMotor, 0.02f); // 角度步长 |  
| |  
| 每 2ms 循环: |  
| Theta += 0.02; |  
| Alg_FOC_Step_OpenLoop(&MyMotor); |  
| TIM1->CCR1 = MyMotor.CCR_a; |  
| TIM1->CCR2 = MyMotor.CCR_b; |  
| TIM1->CCR3 = MyMotor.CCR_c; |  
| |
```

物理意义:

- $V_q_target = 4.0V$ (在 12V 母线电压下, 约 33% 的驱动电压)
- Theta 每 2ms 增加 0.02 rad
- 每秒增加 $0.02 \times 500 = 10$ rad
- 每秒转 $10 / (2\pi) = 1.59$ 转 ≈ 95 RPM
- 速度很慢, 适合初次调试

测试步骤:

- ① 先设 $V_q_target = 0.5V$, 逐步增加到 2V、4V、6V
- ② 每次增加前用万用表测量母线电流 (建议 $< 0.5A$ 空载)
- ③ 观察电机是否平滑旋转
- ④ VOFA+ 观察 3路 CCR 波形是否为马鞍波
- ⑤ 触摸 FD6288Q 和 MOSFET 温度 (温热正常, 烫手异常)

用示波器关键测量:

- FD6288Q 的 HO1/LO1 (U相上下管栅极): 互补 PWM
- U 相输出电压 (对 GND): 平滑的马鞍波 (经过电机绕组电感滤波后)
- 母线电流: 空载 $< 0.5A$ (12V 供电)

转速计算验证:

Theta 在 1 秒内的增量 = $speed \times 500$ (Hz) = elec_rad/s

机械转速 = $elec_rad/s / \text{极对数} / (2\pi) \times 60$ (RPM)

例如 $speed=0.02$, 极对数=7: $0.02 \times 500 / (7 \times 6.28) \times 60 = 13.6$ RPM

调节 V_q_target 的效果:

- V_q_target 越大 → 电机电流越大 → 扭矩越大 → 转速越快 (开环下转速不完全可控)

- V_{q_target} 太小 $\rightarrow$ 无法克服静摩擦力 $\rightarrow$ 电机不动或抖动
- 典型启动电压: 1~3V (对于 12V/2205 电机)

1.2 ADC 电流采样 (3天)

1.2.1 硬件原理

双电阻采样法:

U 相下桥臂 MOSFET 的 Source 脚串联一个采样电阻 (R_{shunt}) 到 GND

V 相下桥臂 MOSFET 的 Source 脚串联一个采样电阻 (R_{shunt}) 到 GND

采样电阻两端的电压 = $I_{phase} \times R_{shunt}$

例如: $I_{phase} = 1A$, $R_{shunt} = 0.01\Omega$ (10m Ω)

采样电压 = $1 \times 0.01 = 0.01V$ (10mV) $\rightarrow$ 太小了!

所以需要 TPA1832 运放放大 (例如放大 50 倍):

输出电压 = $10mV \times 50 = 0.5V \rightarrow$ ADC 可测

运算放大器输出接到 STM32 ADC 引脚:

电机	相电流 ADC	ADC 引脚
电机1	I_U (U相电流)	PC2
电机1	I_V (V相电流)	PC1
电机2	I_U (U相电流)	PC4
电机2	I_V (V相电流)	PC5
电机3	I_U (U相电流)	PA2
电机3	I_V (V相电流)	PA3
电机4	I_U (U相电流)	PA0
电机4	I_V (V相电流)	PA1

1.2.2 ADC 采样时机 (关键!)

PWM 开关瞬间会产生极大的噪声 (MOSFET 导通/关断时的电压尖峰)。

如果在开关瞬间采样, 电流值会完全不可用。

最佳采样时刻 = PWM 周期的中心点 (CCR=0 的下溢点)
在中心对齐模式下:
TIM_CNT: 0 $\rightarrow$ 上升 $\rightarrow$ ARR $\rightarrow$ 下降 $\rightarrow$ 0 $\rightarrow$...
↑↑
└─ 上溢点
└─ 下溢点 (采样!!!)
下溢点: 所有下管都导通 (或即将导通)
此时采样电阻上有电流流过, 且远离开关边沿

1.2.3 CubeMX ADC 配置

打开 CoreFly F2.ioc → ADC1:

```
| ADC1 配置 |  
| |  
| Mode: Independent mode |  
| Resolution: 12-bit (0~4095) |  
| Data Alignment: Right alignment |  
| Scan Conversion: Enable |  
| Continuous: Disable (用定时器触发单次转换) |  
| Discontinuous: Disable |  
| |  
| External Trigger: TIM1_TRGO (定时器1触发) |  
| Trigger Polarity: Rising Edge |  
| |  
| Number of Conversions: 2 (U相 + V相电流) |  
| |  
| Injected Channel 1: ADC_CHANNEL_12 (PC2, I_U) |  
| Sampling Time: 144 Cycles (~8.6μs @ 21MHz) |  
| Injected Channel 2: ADC_CHANNEL_11 (PC1, I_V) |  
| Sampling Time: 144 Cycles |  
| |  
| DMA: Enable (DMA2_Stream0) |  
| Mode: Circular |  
| Data Width: Half Word |  
| |
```

TIM1_TRGO 配置:

TIM1 → Trigger Output (TRGO) → Update Event

这样每次 TIM1 计数到下溢点 (Update Event) 时, 触发一次 ADC 采样。

1.2.4 ADC 电流采样的代码实现

创建新文件: BSP/BSP_Motor.c, BSP/BSP_Motor.h

// ===== BSP_Motor.h 关键内容 =====

```
typedef struct {  
    // --- 定时器硬件句柄 ---  
    TIM_HandleTypeDef *htim; // 指向对应的定时器  
    ADC_HandleTypeDef *hadc; // 指向对应的 ADC  
    // --- PWM 输出引脚 (紧急停机用) ---  
    GPIO_TypeDef *EN_Port;  
    uint16_t EN_Pin;  
    // --- ADC 采样结果 (DMA自动更新) ---  
    volatile uint16_t adc_raw[2]; // [0]=I_U, [1]=I_V  
    // 当我们用多个电机时, 这个是DMA缓冲数组  
    // --- 电流校准参数 (初始化时标定) ---  
    float i_offset_u; // U 相零电流偏置 (ADC 值)
```

```

float i_offset_v; // V 相零电流偏置 (ADC 值)
float i_gain; // ADC → 安培换算系数 (A/ADC_count)

// --- 电流计算结果 (给 FOC 使用) ---
float i_u; // U 相电流 (A)
float i_v; // V 相电流 (A)
float i_w; // W 相电流 (A), 由 KCL 算出:  $i_w = -(i_u + i_v)$ 
} BSP_Motor_t;

// API 函数
void BSP_Motor_Init(BSP_Motor_t *m, TIM_HandleTypeDef *htim,
    ADC_HandleTypeDef *hadc, GPIO_TypeDef *en_port, uint16_t en_pin);
void BSP_Motor_Enable(BSP_Motor_t *m); // EN=1, 启动PWM
void BSP_Motor_Disable(BSP_Motor_t *m); // EN=0, 停止PWM (紧急停机!)
void BSP_Motor_SetPWM(BSP_Motor_t *m, uint32_t ccr_a, uint32_t ccr_b, uint32_t ccr_c);
void BSP_Motor_GetCurrent(BSP_Motor_t *m); // 读取ADC并换算为A
void BSP_Motor_CalibrateOffset(BSP_Motor_t *m); // 标定零偏

// ===== BSP_Motor_CalibrateOffset 实现思路 =====
void BSP_Motor_CalibrateOffset(BSP_Motor_t *m)
{
    // 电机不转时 (PWM 50%占空比), 采样 1000 次取平均
    // 这个平均值就是零电流对应的 ADC 值
    float sum_u = 0, sum_v = 0;
    int N = 1000;
    for(int i = 0; i < N; i++) {
        sum_u += m->adc_raw[0];
        sum_v += m->adc_raw[1];
        vTaskDelay(1); // 1ms
    }
    m->i_offset_u = sum_u / N;
    m->i_offset_v = sum_v / N;
}

// ===== BSP_Motor_GetCurrent 实现 =====
void BSP_Motor_GetCurrent(BSP_Motor_t *m)
{
    // 1. 读取 DMA 缓冲 (已经被 ADC 自动填充)
    uint16_t raw_u = m->adc_raw[0];
    uint16_t raw_v = m->adc_raw[1];

    // 2. 减去零偏
    int16_t diff_u = (int16_t)(raw_u - m->i_offset_u);
    int16_t diff_v = (int16_t)(raw_v - m->i_offset_v);

    // 3. 转换为安培
    //  $i\_gain = 3.3V / 4096 / (\text{运放增益} \times \text{采样电阻})$ 
    // 例如:  $3.3/4096/(50 \times 0.01) = 0.00161 \text{ A/count}$ 

```

```

    m->i_u = diff_u * m->i_gain;
    m->i_v = diff_v * m->i_gain;
    m->i_w = -(m->i_u + m->i_v); // KCL
}

```

1.2.5 电流增益 (i_gain) 的计算公式

假设硬件参数:

$V_{ref} = 3.3V$ (ADC 参考电压)

ADC_bits = 12 (4096 counts)

运放增益 = Gain (例如 TPA1832 典型 50 V/V)

采样电阻 = R_{shunt} (例如 0.01 Ω)

```

| |
| 每 count 对应的电压:  $V_{per\_cnt} = V_{ref} / 4096$  |
| =  $3.3 / 4096 = 0.00080566 V$  |
| |
| 运放输入电压:  $V_{input} = V_{output} / Gain$  |
| 采样电阻电流:  $I_{phase} = V_{input} / R_{shunt}$  |
| |
| 综合:  $I_{phase} = V_{per\_cnt} \times ADC\_count / (Gain \times R_{shunt})$  |
| |
|  $i\_gain = V_{ref} / 4096 / (Gain \times R_{shunt}) [A/count]$  |
| |
| 数值示例:  $i\_gain = 3.3/4096/(50 \times 0.01) = 0.00161 A/cnt$  |
| 满量程  $\pm 2A$  对应 ADC count 约  $\pm 1240$  |
| |

```

1.2.6 验证方法

- ① 先不接电机, PWM 输出 50% 占空比
- ② 运行 `BSP_Motor_CalibrateOffset()`, 记录 `offset_u`, `offset_v`
- ③ 在 VOFA+ 上打印原始 ADC 值和换算后的电流值
- ④ 用手轻轻转动电机 (此时是一个发电机): 应该能看到电流波动
- ⑤ 接上电机, 运行开环 SVPWM, VOFA+ 看 I_u , I_v 波形:
 - 应该是两个相位差 120° 的正弦波
 - 电流幅值随 V_q_target 增大而增大
 - 波形应该平滑, 没有大的毛刺

验证标准:

- ☐ 电机不转时, $I_u \approx I_v \approx 0A$ (零偏已校准)
- ☐ 开环驱动时, VOFA+ 看到两相正弦电流, 相位差约 120°
- ☐ 电流幅值与 V_q_target 成正比
- ☐ 电流波形无明显毛刺或跳变

1.3 通用 PID 控制器 (2天)

1.3.1 PID 基本公式

位置式 PID (用于电流环、速度环):

```
| |
| u(k) = Kp × e(k) |
| + Ki × T_s × Σe(i) ← 积分项 |
| + Kd × [e(k) - e(k-1)] / T_s ← 微分项 |
| |
| 其中: e(k) = target - feedback (误差) |
| T_s = 控制周期 (秒) |
| u(k) = 控制器输出 |
| |
```

增量式 PID (用于角度环/位置环):

```
| |
| Δu(k) = Kp × [e(k) - e(k-1)] |
| + Ki × T_s × e(k) |
| + Kd × [e(k) - 2e(k-1) + e(k-2)] / T_s |
| |
| u(k) = u(k-1) + Δu(k) |
| |
```

1.3.2 PID 必须集成的保护功能

```
| 功能 | 说明 | | |
|---|---|---|---|
| 输出限幅 | |u| ≤ out_max, 防止积分饱和后输出失控 |
| 积分分离 | 当 |error| > threshold 时清零积分 |
| | (大误差时不积分, 避免超调) |
| 抗积分饱和 | 输出被钳位时停止积分 (Clamping) |
| | 条件: 如果 (u == out_max && error > 0) |
| | 或 (u == out_min && error < 0) |
| | 则停止积分累加 |
| 微分先行 | 微分项对 feedback 微分而非 error |
| | (避免目标突变导致的微分冲击) |
| 死区 | |error| < deadband → 输出维持不变 |
| | (减少微振动) |
```

1.3.3 PID 结构体设计

// ===== Alg_PID.h 关键内容 =====

```
typedef struct {
    // --- 参数 (用户设定) ---
    float Kp;
    float Ki;
    float Kd;
    float Ts; // 控制周期 (秒)
    // --- 限幅 ---
```

```

float out_max; // 输出上限
float out_min; // 输出下限
float integral_max; // 积分上限
// --- 功能开关 ---
uint8_t enable_integral_separation; // 积分分离
float integral_sep_threshold; // 分离阈值
uint8_t enable_anti_windup; // 抗饱和
uint8_t enable_deriv_on_meas; // 微分先行
float deadband; // 死区
// --- 内部状态 (私有) ---
float integral;
float prev_error;
float prev_feedback;
float output;
} PID_t;

void PID_Init(PID_t *pid, float kp, float ki, float kd, float ts,
              float out_max, float out_min);
float PID_Calc(PID_t *pid, float target, float feedback);
void PID_Reset(PID_t *pid); // 清空积分和历史
void PID_SetParam(PID_t *pid, float kp, float ki, float kd);

```

1.3.4 PID 单元测试

写一个纯软件测试 (在 PC 或用 Keil 的 simulator):

测试用例1: 阶跃响应

目标值 = 1.0, 初始反馈 = 0.0

$K_p=1$, $K_i=10$, $K_d=0$, $T_s=0.001$

预期: 反馈值逐渐趋近 1.0, 最终稳态误差 ≈ 0

测试用例2: 输出限幅

$out_max = 0.5$, 目标值 = 10.0

预期: 输出不会超过 0.5

测试用例3: 积分分离

$error > threshold$ 时, 积分不累加

预期: 超调量比不做分离时小

1.4 SMO + PLL 观测器实现 (4天)

这是整个 FOC 开发中最有挑战性的部分。

1.4.1 需要预先测量的电机参数

在开始 SMO 开发前, 必须先知道电机的相电阻 R 和相电感 L :

| 测量相电阻 R : |

| |

| 方法: 使用万用表测量任意两相之间的电阻 |
 | $R_{line} = U/V$ 间电阻 |
 | 对于星形接法: $R_{phase} = R_{line} / 2$ |
 | |
 | 典型值: 2205 电机 $\approx 0.1 \sim 0.3 \Omega$ |
 | |
 | □ 注意: 万用表测小电阻不准, 建议用数字电桥 |
 | 或者: 给任意两相通直流电 (例如 1A), 测电压降 |
 | |
 | _____ |
 | |
 | 测量相电感 L: |
 | |
 | 方法: 使用 LCR 表, 频率设 1kHz ~ 10kHz |
 | 典型值: 2205 电机 $\approx 10 \sim 50 \mu H$ |
 | |
 | 无 LCR 表时的粗略估算法: |
 | $L \approx V_{dc} \times T_{pwm} / (2 \times I_{ripple})$ |
 | 其中: V_{dc} =母线电压, T_{pwm} =PWM周期, I_{ripple} =电流纹波 |
 | |
 | _____ |

1.4.2 SMO 代码结构

在 Alg_FOC_Math.h 中新增 SMO 结构体:

// ===== 新增到 Alg_FOC_Math.h =====

```
typedef struct {
    // --- 电机参数 ---
    float R; // 相电阻 ( $\Omega$ )
    float L; // 相电感 (H)

    // --- SMO 参数 ---
    float K_slide; // 滑模增益
    float delta; // 饱和函数边界层
    float tau_lpf; // BEMF 低通滤波时间常数

    // --- PLL 参数 ---
    float Kp_pll; // PLL 比例增益
    float Ki_pll; // PLL 积分增益

    // --- 内部状态 ---
    float I_hat_alpha; // 估算电流  $\alpha$ 
    float I_hat_beta; // 估算电流  $\beta$ 
    float E_alpha; // 估算反电动势  $\alpha$  (低通滤波后)
    float E_beta; // 估算反电动势  $\beta$  (低通滤波后)
    float theta_pll; // PLL 估算角度 (弧度)
    float omega_pll; // PLL 估算电角速度 (rad/s)
    float integral_pll; // PLL 积分项
} SMO_Observer_t;

// 步骤函数 (每次 FOC 循环调用一次)
void SMO_Update(SMO_Observer_t *smo, float V_alpha, float V_beta,
```



```
float I_alpha, float I_beta, float Ts);
```

1.4.3 SMO 核心函数实现

```
// ===== Alg_FOC_Math.c 新增 (SMO部分) =====  
void SMO_Update(SMO_Observer_t *smo, float V_alpha, float V_beta,  
float I_alpha, float I_beta, float Ts)  
{  
    // 1. 电流误差  
    float err_alpha = smo->I_hat_alpha - I_alpha;  
    float err_beta = smo->I_hat_beta - I_beta;  
  
    // 2. 饱和函数  
    float z_alpha = SMO_Sat(err_alpha, smo->K_slide, smo->delta);  
    float z_beta = SMO_Sat(err_beta, smo->K_slide, smo->delta);  
  
    // 3. 更新电流估算  
    float inv_L = 1.0f / smo->L;  
    float R_div_L = smo->R * inv_L;  
    smo->I_hat_alpha += Ts * (-R_div_L * smo->I_hat_alpha  
    + inv_L * V_alpha  
    - inv_L * z_alpha);  
    smo->I_hat_beta += Ts * (-R_div_L * smo->I_hat_beta  
    + inv_L * V_beta  
    - inv_L * z_beta);  
  
    // 4. 低通滤波提取 BEMF  
    float lpf_coef = Ts / smo->tau_lpf;  
    smo->E_alpha += lpf_coef * (z_alpha - smo->E_alpha);  
    smo->E_beta += lpf_coef * (z_beta - smo->E_beta);  
  
    // 5. PLL  
    float sin_pll = sinf(smo->theta_pll);  
    float cos_pll = cosf(smo->theta_pll);  
    // 相位误差: 用 BEMF 矢量叉乘  
    float pll_err = -smo->E_alpha * cos_pll - smo->E_beta * sin_pll;  
    smo->integral_pll += pll_err * Ts;  
    // 抗积分饱和  
    float integral_max = 5000.0f; // 最大电角速度 (rad/s)  
    if (smo->integral_pll > integral_max) smo->integral_pll = integral_max;  
    if (smo->integral_pll < -integral_max) smo->integral_pll = -integral_max;  
    smo->omega_pll = smo->Kp_pll * pll_err + smo->Ki_pll * smo->integral_pll;  
    smo->theta_pll += smo->omega_pll * Ts;  
  
    // 6. 角度归一化到 [-PI, PI]  
    if (smo->theta_pll > M_PI) smo->theta_pll -= 2.0f * M_PI;  
    if (smo->theta_pll < -M_PI) smo->theta_pll += 2.0f * M_PI;  
}
```

```
// 饱和函数
static inline float SMO_Sat(float err, float K, float delta)
{
    if (err > delta) return K;
    if (err < -delta) return -K;
    return K * err / delta; // 线性区
}
```

1.4.4 SMO 调试方法

第一步: 开环验证 SMO

1. 电机运行在开环强拖模式 (V_q target 固定, θ 手动递增)
2. 开环给了 θ → 这是我们已知的"真值"
3. SMO 也在同时估算角度 θ_{pll}
4. 在 VOFA+ 上同时画两条线:
 - 线1 (蓝): 开环 θ (已知真值)
 - 线2 (红): SMO 估算 θ_{pll}
5. 如果 SMO 正确, 两条线应该重叠 (估算角度跟踪上真实角度)
6. 如果 SMO 发散 (两条线分开越来越远) → 检查 K_{slide} 是否太小、电机参数是否填对

第二步: 调节参数

- 如果角度估算抖动大: 增大 δ (让饱和函数更平滑)
- 如果角度收敛慢: 增大 K_{slide}
- 如果角度有静差: 增大 PLL 的 K_{i_pll}
- 如果角度滞后明显: 减小 τ_{lpf} (降低滤波强度)

第三步: 速度估算验证

VOFA+ 画 ω_{pll} , 与 θ 的变化率比较。

开环下 $speed=0.02 \rightarrow d\theta/dt = 0.02/0.002 = 10 \text{ rad/s}$ (电角速度)

ω_{pll} 应该约等于 10 rad/s

1.4.5 SMO 初始化示例参数

以 2205-2300KV 电机 (极对数=7) 为例:

```
smo->R = 0.15f; // 相电阻 (实测)
smo->L = 0.000025f; // 相电感 25μH (实测)
smo->K_slide = 30.0f; // 滑模增益 (初始值, 需要调)
smo->delta = 1.5f; // 边界层
smo->tau_lpf = 0.001f; // BEMF 滤波 1ms
smo->Kp_pll = 400.0f; // PLL 带宽 200 rad/s (BW=200, Kp=2*BW)
smo->Ki_pll = 40000.0f; // Ki = BW^2 = 200^2 = 40000
```

=====

1.5 电流环闭环 (3天)

=====

1.5.1 电流环架构

$I_d=0$ 控制 (表贴式 PMSM 的标准策略):

```

目标 Id = 0  ———→ [PID_Id] ———→ Vd_target ———┐
      │                                     │
      └——→ 逆Park → SVPWM
目标 Iq = throttle ———→ [PID_Iq] ———→ Vq_target ———┐
      │                                     │
      └——→ 逆Park → SVPWM
电流环执行频率: 20kHz (与 PWM 频率同步)

```

1.5.2 电流环完整代码流程

```
// ===== 每次电流环迭代 (放在 FOC 20kHz 中断或任务中) =====

void FOC_CurrentLoop_Step(FOC_Motor_t *motor, SMO_Observer_t *smo, PID_t *pid_id, PID_t *pid_iq)
{
    // 1. ADC 采样完成 → 读取相电流
    float I_a = motor->I_a; // 由 BSP_Motor_GetCurrent() 更新
    float I_b = motor->I_b;
    float I_c = -(I_a + I_b);

    // 2. Clarke 变换
    Alg_FOC_Clarke(motor);

    // 3. Park 变换 (使用 SMO+PLL 估算的角度)
    motor->Theta = smo->theta_pll;
    Alg_FOC_Park(motor);

    // 现在 motor->I_d 和 motor->I_q 是有意义的直流量

    // 4. 电流环 PID
    // Id 目标永远为 0 (不励磁)
    // Iq 目标由飞控姿态环给出, 或手动设置用于测试
    motor->V_d_target = PID_Calc(pid_id, 0.0f, motor->I_d);
    motor->V_q_target = PID_Calc(pid_iq, Target_Iq, motor->I_q);

    // 5. 逆 Park → SVPWM → 更新 CCR
    Alg_FOC_InvPark(motor);
    Alg_FOC_SVPWM(motor);
}
```

1.5.3 电流环 PID 参数整定方法

使用经典的 Ziegler-Nichols 或手动试凑法:

- | Id 环 PID (典型初始值): |
- | $K_p = 0.5 \sim 2.0$ |
- | $K_i = 100 \sim 500$ |
- | $K_d = 0$ (电流环一般不需要微分) |
- | $T_s = 0.00005$ (20kHz) |
- | $out_max = V_Bus \times 0.9$ (最大输出电压) |
- | $out_min = -V_Bus \times 0.9$ |
- | |
- | Iq 环 PID (典型初始值): |
- | 与 Id 环参数相同 (电气对称) |
- | $out_max = V_Bus \times 0.9$ |
- | $out_min = -V_Bus \times 0.9$ |

整定步骤:

- ① 先调 Kp: 从 0.5 开始逐步增加, 直到 I_q 对阶跃响应有轻微超调
- ② 再加 Ki: 从 100 开始逐步增加, 直到稳态误差在 10ms 内消除
- ③ 观察 V_q target: 如果 V_q 在饱和值附近剧烈跳变 $\rightarrow$ Kp 太大
- ④ 观察 I_q : 如果 I_q 有大幅过冲 $\rightarrow$ 减小 Kp, 增大 Ki

1.5.4 闭环验证标准

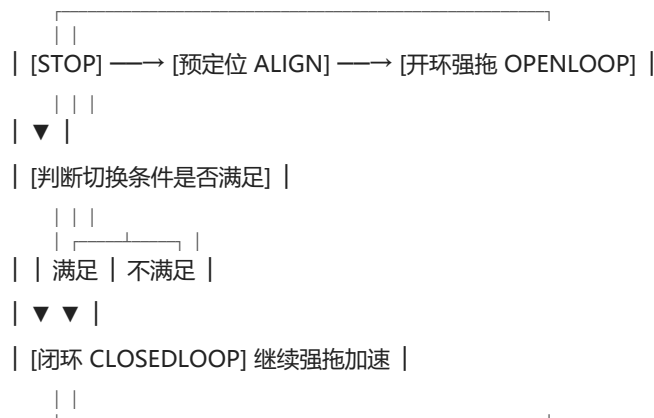
- VOFA+ 上 I_q 能跟踪目标值 (例如给 $I_{q_target} = 0.5A$, 实际 $I_q \approx 0.5A$)
- I_d 接近 0 (稳态误差 $< 0.05A$)
- V_q target 不在饱和值附近剧烈跳变
- 电机平稳旋转, 无抖动、无卡顿、无异常噪声
- 突加负载 (用手轻捏电机轴): I_q 迅速上升以维持转速, I_q 稳态误差小

1.6 电机启动流程 (3天)

无感 FOC 最难点: 启动!

静止时没有反电动势, SMO 无法估算角度 $\rightarrow$ 不能直接闭环。

使用三段式启动策略:



1.6.1 阶段1: 预定位 (Pre-Align)

目的: 让转子转到已知位置, 为开环启动提供准确的初始角度。

方法: 给定一个固定的电角度 (例如 $\theta=0^\circ$) + 一个小电流 (V_q), 保持一段时间。

代码:

```
// 进入预定位
motor->Mode = MOTOR_MODE_ALIGN;
align_theta = 0.0f; // 对齐到 0°
align_vq = 1.5f; // 较小的驱动电压
align_duration = 0.5f; // 保持 0.5 秒
// 强制 Theta = 0, Vq = 1.5V, Vd = 0
// 转子会在电磁力的作用下旋转到与定子磁场对齐的位置
```

1.6.2 阶段2: 开环强拖 (Open-Loop Ramp)

目的: 线性增加角度 (相当于线性加速旋转磁场), 拖着转子一起转。

```
// 进入开环强拖
motor->Mode = MOTOR_MODE_OPEN_LOOP;
ramp_start_vq = 2.0f;
ramp_end_vq = 4.0f;
ramp_start_speed = 5.0f; // 起始电角速度 (rad/s)
ramp_end_speed = 300.0f; // 终止电角速度 (rad/s)
ramp_duration = 2.0f; // 加速持续时间 (秒)

// 每个控制周期:
// speed = ramp_start_speed + (ramp_end_speed - ramp_start_speed) × (t / ramp_duration)
// Vq = ramp_start_vq + (ramp_end_vq - ramp_start_vq) × (t / ramp_duration)
// theta += speed × dt
// Alg_FOC_Step_OpenLoop(&motor);
```

判断条件 (三个都满足才能切):

- ||
- | 条件1: 估算速度 > 最小切换速度 |
- | $\text{smo} \rightarrow \omega_{\text{pll}} > \text{MIN_SWITCH_SPEED}$ (例如 150 rad/s) |
- ||
- | 条件2: 估算角度误差小 |
- | 开环角度与 SMO 估算角度的差值 < MAX_ANGLE_ERR |
- | $|\text{openloop_theta} - \text{smo} \rightarrow \theta_{\text{pll}}| < 0.3 \text{ rad } (\sim 17^\circ)$ |
- ||
- | 条件3: 电流稳定 |
- | i_q 的波动幅度 < 最大允许值 |
- | 例如: $|\dot{i}_q - i_{q_filtered}| < 0.2 \text{ A}$ |
- ||
- | 三个条件同时满足且持续约 100ms 后, 才能切入闭环 |
- ||

切入闭环的动作:

- ① 角度来源: 开环累加值 $\rightarrow$ SMO+PLL 估算值 (需要平滑过渡!)
- ② 控制模式: 电压开环 $\rightarrow$ 电流闭环 (V_d_target 和 V_q_target 由 PID 产生)
- ③ PID 预热: 切入前预装 PID 积分值 = 当前的 V_d_target/V_q_target 平均值

```
// 用加权平均过渡，而不是直接跳变
// 过渡期内 (例如 50ms):
// used_theta = (1 -  $\alpha$ )  $\times$  openloop_theta +  $\alpha$   $\times$  smo_theta
//  $\alpha$  从 0 线性增加到 1
```

```
// ===== App Motor Ctrl.c 新增状态机 =====
```

```
typedef enum {
```

```

MOTOR_STATE_STOP = 0, // 停机
MOTOR_STATE_ALIGN, // 预定位
MOTOR_STATE_OPENLOOP, // 开环强拖
MOTOR_STATE_SWITCHING, // 切换过渡中
MOTOR_STATE_CLOSEDLOOP, // 正常闭环
MOTOR_STATE_FAULT, // 故障
} Motor_State_e;

typedef struct {
    // ... (继承现有 Motor_Ctrl_t 的字段)

    Motor_State_e State;
    uint32_t state_entry_time; // 进入当前状态的时间 (ms)
    float openloop_theta; // 开环累加角度
    float align_theta; // 预定位目标角度

    // 开环加速参数
    float ramp_start_speed;
    float ramp_end_speed;
    float ramp_start_vq;
    float ramp_end_vq;
    float ramp_duration;

    // 切换参数
    float switch_alpha; // 角度混合系数
    float switch_duration; // 切换过渡时间 (s)
} Motor_Ctrl_Full_t;

void Motor_StateMachine_Run(Motor_Ctrl_Full_t *m, SMO_Observer_t *smo)
{
    switch(m->State)
    {
    case MOTOR_STATE_STOP:
        // 所有 PWM 输出 50% (零矢量)
        break;

    case MOTOR_STATE_ALIGN:
        // 固定角度 + 固定 Vq, 计时
        m->MathParam.Theta = m->align_theta;
        m->MathParam.V_q_target = 1.5f;
        Alg_FOC_Step_OpenLoop(&m->MathParam);
        if (elapsed > ALIGN_TIME_MS) {
            m->State = MOTOR_STATE_OPENLOOP;
            m->state_entry_time = current_time;
            m->openloop_theta = m->align_theta;
        }
        break;

    case MOTOR_STATE_OPENLOOP:

```

```

// 线性加速
float progress = elapsed / m->ramp_duration;
float speed = m->ramp_start_speed + (m->ramp_end_speed - m->ramp_start_speed) * progress;
float Vq = m->ramp_start_vq + (m->ramp_end_vq - m->ramp_start_vq) * progress;
m->openloop_theta += speed * Ts;
m->MathParam.Theta = m->openloop_theta;
m->MathParam.V_q_target = Vq;
Alg_FOC_Step_OpenLoop(&m->MathParam);
// 检查切换条件
if (CheckSwitchCondition(m, smo)) {
    m->State = MOTOR_STATE_SWITCHING;
    m->state_entry_time = current_time;
}
break;

case MOTOR_STATE_SWITCHING:
    // 角度平滑过渡
    m->switch_alpha = elapsed / m->switch_duration; // 0 → 1
    if (m->switch_alpha >= 1.0f) {
        m->State = MOTOR_STATE_CLOSEDLOOP;
    }
    // 混合角度
    m->MathParam.Theta = (1 - m->switch_alpha) * m->openloop_theta
    + m->switch_alpha * smo->theta_pll;
    // 仍然开环给 Vq (过渡期间暂不切入电流环)
    Alg_FOC_Step_OpenLoop(&m->MathParam);
    break;

case MOTOR_STATE_CLOSEDLOOP:
    m->MathParam.Theta = smo->theta_pll; // 完全使用估算角度
    Alg_FOC_Step_ClosedLoop(&m->MathParam);
    break;

case MOTOR_STATE_FAULT:
    // 紧急停机
    BSP_Motor_Disable(&motor1);
    break;
}
}

```

1.6.6 常见启动问题排查

现象 可能原因与解决方法	
预定位时电机抖动	Vq 太大 → 减小 对齐时间不够
强拖时电机失步(异响)	速度增长太快 → 减慢加速度

```
| | Vq 太小 → 增加 Vq |
|-----|
| 切入闭环时电机骤停 | SMO 角度估算不准 → 先开环验证SMO |
| | PID 参数太激进 → 减小 Kp |
|-----|
| 闭环后电机抖动/啸叫 | PID 参数太大 → 减小 Kp |
| | SMO 角度抖 → 增大 LPF 滤波 |
|-----|
```

第2阶段: 传感器与姿态解算 (预计 5-7 天)

前提: 第1阶段已完成, 电机1 可以闭环稳定运行。

2.1 补全 SPL06-001 气压计驱动 (1天)

文件: BSP/BSP_SPL06.c (已有骨架)

实现要点:

- I2C 读取校准系数 (18字节, 地址 0x10 ~ 0x21)
- 校准系数解析 (按 SPL06 datasheet 的公式)
- 读取原始温度 (0x03~0x05) 和气压 (0x00~0x02)
- 温度补偿后计算气压值 (Pa)
- 气压→高度换算 (标准大气压公式)

高度换算公式:

```
| |
| h = 44330 × [1 - (P / P0)^(1/5.255)] |
| |
| 其中: P = 当前气压 (Pa) |
| P0 = 海平面标准气压 (101325 Pa) |
| h = 高度 (m) |
| |
| 注意: 这个公式假设标准大气条件, 实际精度约 ±1m |
| 需要起飞前标定 P0 (取地面气压为参考) |
| |
```

2.2 姿态解算 Mahony 互补滤波 (3天)

2.2.1 四元数基础

使用四元数表示姿态 (避免欧拉角的万向节死锁问题):

四元数 $q = [q_0, q_1, q_2, q_3]$

其中: $q_0 = \cos(\theta/2)$ (标量部分)

$q_1 = \sin(\theta/2) \times u_x$ (矢量部分 x)

$q_2 = \sin(\theta/2) \times u_y$ (矢量部分 y)

$q_3 = \sin(\theta/2) \times u_z$ (矢量部分 z)

四元数 → 欧拉角:

```
| |
```



```

| Roll = atan2(2·(q0·q1 + q2·q3), 1-2·(q1²+q2²)) |
| Pitch = arcsin(2·(q0·q2 - q3·q1)) |
| Yaw = atan2(2·(q0·q3 + q1·q2), 1-2·(q2²+q3²)) |
| |
|_____|

```

2.2.2 Mahony 互补滤波算法

核心思想:

- 陀螺仪: 动态响应快 (高频准), 但积分会漂移 (低频不准)
- 加速度计: 静态准确 (低频准, 能测重力方向), 但动态噪声大 (高频不准)
- 互补滤波: 陀螺仪积分 + 加速度计修正 = 取两者之长

```

| Mahony 算法步骤 (每次循环): |
| |
| 1. 加速度归一化: |
| norm = sqrt(ax²+ay²+az²) |
| ax/=norm, ay/=norm, az/=norm |
| |
| 2. 从当前四元数推算重力方向 (理论值): |
| gx_theory = 2·(q1·q3 - q0·q2) |
| gy_theory = 2·(q0·q1 + q2·q3) |
| gz_theory = q0² - q1² - q2² + q3² |
| |
| 3. 叉积 = 实际加速度 × 理论重力 = 角度误差: |
| ex = ay · gz_theory - az · gy_theory |
| ey = az · gx_theory - ax · gz_theory |
| ez = ax · gy_theory - ay · gx_theory |
| |
| 4. PI 修正陀螺仪: |
| gyro_x += Kp · ex + Ki · ∫ex |
| gyro_y += Kp · ey + Ki · ∫ey |
| gyro_z += Kp · ez + Ki · ∫ez |
| |
| 5. 四元数更新 (一阶龙格库塔): |
| q0 += 0.5·(-q1·gx - q2·gy - q3·gz) · dt |
| q1 += 0.5·( q0·gx + q2·gz - q3·gy) · dt |
| q2 += 0.5·( q0·gy - q1·gz + q3·gx) · dt |
| q3 += 0.5·( q0·gz + q1·gy - q2·gx) · dt |
| |
| 6. 四元数归一化 |
| |
|_____|

```

Mahony 参数 (典型值):

$K_p = 2.0$ (比例增益)

$K_i = 0.0$ (积分增益, 可以先不加, 避免偏航持续漂移时积分发散)

验证方法:

- ① 板子水平放置 → Roll $\approx 0^\circ$, Pitch $\approx 0^\circ$
- ② 板子前倾 45° → Pitch $\approx +45^\circ$

- ③ 板子右倾 $45^\circ \rightarrow \text{Roll} \approx +45^\circ$
- ④ 快速晃动板子 $\rightarrow$ 姿态角平滑跟随, 无明显延迟或过冲

2.3 传感器融合任务 App_Sensor (1天)

FreeRTOS 任务: Sensor_Task, 1kHz, 优先级最高 (6)

职责:

- ① 读取 BMI160 (加速度+角速度)
- ② 读取 SPL06 (气压+温度) — 100Hz 即可
- ③ 调用 Mahony 姿态解算
- ④ 写入全局状态结构体

// 全局状态 (所有任务共享, 用互斥锁或队列保护)

```
typedef struct {  
    // 姿态 (由 Sensor_Task 更新)  
    float roll, pitch, yaw; // 欧拉角 (rad)  
    float q0, q1, q2, q3; // 四元数  
    float gyro_x, gyro_y, gyro_z; // 角速度 (rad/s)  
    float acc_x, acc_y, acc_z; // 加速度 (m/s2)  
    // 高度 (由 Sensor_Task 更新)  
    float altitude; // 相对高度 (m)  
    float baro_pressure; // 气压 (Pa)  
    // 电池 (由 Monitor_Task 更新)  
    float vbat; // 电池电压 (V)  
} FlightState_t;
```

第3阶段: 四电机扩展与飞控核心 (预计 7-10 天)

前提:

- 电机1 闭环稳定运行 (Iq 跟踪正常)
- 姿态解算准确 (Roll/Pitch 误差 $< 2^\circ$)
- BMI160 数据稳定

3.1 将 App_Motor_Ctrl 扩展为 4 电机 (2天)

把 Motor_Ctrl_t motor[4] 数组化:

// ===== 改进后的架构 =====

Motor_Ctrl_Full_t g_motor[4]; // 4 个电机控制器

PID_t g_pid_id[4], g_pid_iq[4]; // 每个电机各一套电流环 PID

SMO_Observer_t g_smo[4]; // 每个电机各一个 SMO

// FreeRTOS 任务: Motor_Ctrl_Task @ 20kHz, 优先级 5

void Motor_Ctrl_Task(void *pv)

```
{  
    while(1) {
```

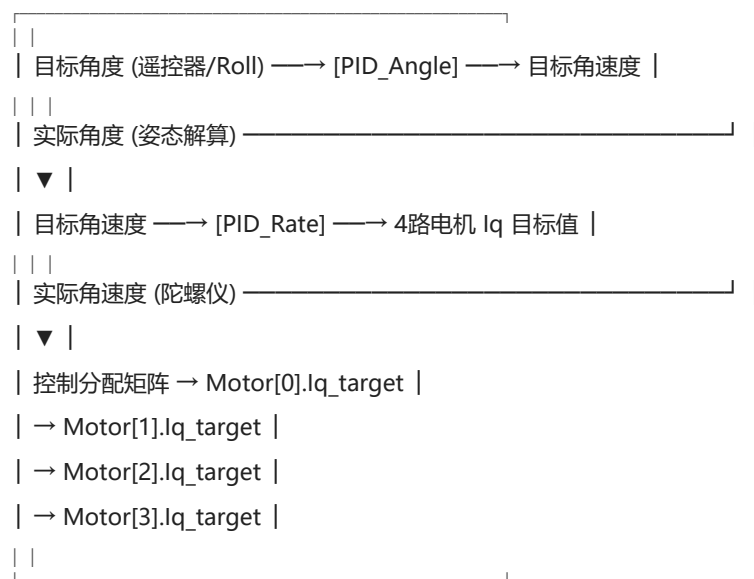
```

    for(int i = 0; i < 4; i++) {
        // 1. 读取该电机的电流 ADC
        // 2. 运行 SMO 更新角度
        // 3. 运行电流环 PID
        // 4. 逆 Park → SVPWM → 更新该路 PWM CCR
    }
    vTaskDelayUntil(...);
}
}

```

3.2 姿态控制任务 (3天)

串级 PID 架构:



串级 PID 的优势:

- 内环 (角速度环) 运行频率高 (1kHz), 快速抑制扰动
- 外环 (角度环) 运行频率可稍低 (500Hz), 提供角度跟踪
- 两层 PID 的带宽可以分别调节

角度环 PID:

输入: 目标角度 (roll_target, pitch_target, yaw_target)

反馈: 实际角度 (roll, pitch, yaw)

输出: 目标角速度 (gyro_x_target, gyro_y_target, gyro_z_target)

PID: 纯 P 或 PI (不要 D, 角度信号缓慢不需要微分)

频率: 500Hz

角速度环 PID:

输入: 目标角速度 (来自角度环输出)

反馈: 实际角速度 (直接来自陀螺仪, 无需滤波)

输出: Roll_Out, Pitch_Out, Yaw_Out

PID: PID (D 项可以帮助抑制振动)

频率: 1kHz

控制分配矩阵 (X 型四旋翼):

```

| |
| 设 Thrust=T (总推力), Roll=R, Pitch=P, Yaw=Y |
| |
| Motor1 (前右, CW): +T -R -P +Y |
| Motor2 (后右, CCW): +T -R +P -Y |
| Motor3 (后左, CW): +T +R +P +Y |
| Motor4 (前左, CCW): +T +R -P -Y |
| |
| CW = 顺时针桨, CCW = 逆时针桨 |
| |
| 实际代码: |
| motor[0].Iq_target = throttle + roll_out - pitch_out + yaw_out |
| motor[1].Iq_target = throttle - roll_out + pitch_out - yaw_out |
| motor[2].Iq_target = throttle + roll_out + pitch_out + yaw_out |
| motor[3].Iq_target = throttle - roll_out - pitch_out - yaw_out |
| |
| 输出限幅: Iq_target ∈ [0, MAX_IQ] |
| (电机不能反向旋转, Iq 不能为负) |
| |

```

3.3 地面测试 (2天)

不装桨! 安全第一!

测试步骤:

- ① 4 个电机全部进入闭环模式
- ② 拿起飞控板, 手动倾斜:
 - 前倾 (Pitch正): 前电机减速, 后电机加速
 - 右倾 (Roll正): 右电机减速, 左电机加速
 - 逆时针转: CW电机加速, CCW电机减速
- ③ VOFA+ 观察 4 路 Iq 值是否正确响应
- ④ 观察 PID 输出: 没有剧烈震荡、没有饱和

第4阶段: 通信与安全监控 (预计 4-6 天)

4.1 LoRa 驱动 (2天)

文件: BSP/BSP_Ra-01SC.c (当前为空)

基于 SX1278 的 SPI 驱动:

- 寄存器读写封装 (SPI3)
- 初始化: 频率 433MHz, SF=7, BW=125kHz, CR=4/5
- 发送/接收函数
- 中断处理 (DIO0 映射到 EXTI9_5)

4.2 遥控数据解析 (2天)

文件: Apps/App_Comm.c (新建)

职责:

- ① 接收遥控器数据帧 (LoRa 接收)
- ② 解析: Roll/Pitch/Yaw/Throttle/AUX 通道值
- ③ 打包遥测数据回传 (姿态/高度/电池/RSSI)
- ④ 丢包检测与 failsafe 触发

4.3 状态监控任务 (1天)

文件: Apps/App_Monitor.c (新建)

职责:

- ① 电池电压检测 (ADC2 读取 VBAT_SENSE)
- ② 低电压报警 ($\text{VBAT} < 3.5\text{V}/\text{cell}$ → LED 闪烁 + 蜂鸣器)
- ③ 低电压自动降落 ($\text{VBAT} < 3.3\text{V}/\text{cell}$ → 强制降落)
- ④ 通信超时检测 (遥控信号丢失 $> 1\text{s}$ → failsafe)
- ⑤ LED 状态指示:
 - 初始化中: 红灯慢闪
 - 待命 (ARMED): 蓝灯常亮
 - 飞行中: 蓝灯快闪
 - 低电量: 红灯快闪
 - 故障: 红灯常亮 + 蜂鸣器

第5阶段: 系统集成与飞行测试 (预计 5-7 天)

□ 此阶段的所有测试都必须在安全的室外空旷场地进行!

5.1 系留飞行测试

准备工作:

- 用绳子将无人机固定在地面 (绳长 1-2m, 限制飞行范围)
- 设置电流保护 (母线电流 $> 10\text{A}$ → 立即停机)
- 配置上锁/解锁逻辑 (两个摇杆内八字 → 解锁)
- 设置紧急停机开关 (遥控器 AUX 通道)

测试序列:

- ① 解锁 → 怠速 (4 个电机以最低速度转, 桨不产生升力)
- ② 缓慢推油门 → 观察离地 → 保持在 30cm 高度悬停
- ③ 轻微打杆 → 观察无人机响应是否正确
- ④ 松开摇杆 → 观察能否自稳回中
- ⑤ 收油门 → 降落 → 上锁

5.2 PID 整定 (飞行中)

姿态 PID 飞行整定顺序:

- ① 先整定角速度环 (Rate PID)

- 增大 P 直到出现轻微振荡 → 退回 60%
 - 增大 D 直到振荡消失
 - 增大 I 直到稳态偏差消除
- ② 再整定角度环 (Angle PID)
- 增大 P 直到姿态跟踪快速 (没有明显延迟)
 - 一般不需要 I 和 D

5.3 自由飞行测试

- ① 低空悬停测试 (1m 高度, 测试稳定性)
- ② 手动飞行测试 (前进/后退/左右/旋转)
- ③ 高度保持测试 (气压计定高)
- ④ 自动返航测试 (failsafe 触发)
- ⑤ 电池续航测试 (记录飞行时间, 计算效率)

=====

====

附 录

=====

====

附录A：PID 参数整定方法（经验法）

A.1 电流环 PID (最内层, 最快)

- 整定目标: Iq 对阶跃指令的响应快 (< 5ms) 且无过冲
- 步骤:
- 1. Ki=0, Kd=0, 逐步增加 Kp
 - 2. 当 Iq 开始有过冲 → 退回 Kp 的 60%
 - 3. 逐步增加 Ki, 直到稳态误差在 10ms 内归零
 - 4. Kd 一般不需要 (电流信号噪声大, 微分反而有害)
- 参考范围 (2205 电机, 20kHz):
- Kp: 0.5 ~ 3.0
- Ki: 50 ~ 300

A.2 速度环 PID (中层, 可选)

- 整定目标: 速度跟踪平滑, 负载突变后快速恢复
- 步骤:
- 1. 同电流环, 先 P 后 I
 - 2. Kd 可以加一点 (速度信号比电流信号干净)

A.3 姿态角速度环 PID (中层)

- 整定目标: 对遥控器输入响应快, 抗风扰动强
- 步骤:
- 1. 悬停状态下从 0 逐步增大 P

2. 直到飞机出现高频小幅度振荡 → 此时的 P 是临界值
3. P 设为临界值的 50~60%
4. 加 D 抑制振荡 ($D = P/50 \sim P/20$)
5. 加 I 消除稳态偏差 (长期悬停时飞机不慢慢漂移)

A.4 姿态角度环 PID (外层)

整定目标: 给定角度指令后飞机平稳跟踪, 无大幅超调

步骤:

1. P 从 2 开始逐步增加
2. 快速打杆后飞机应迅速跟随, 不振荡
3. 一般不需要 I 和 D (角速度环已经有 I 了)

附录B: 电机参数测量方法 (动手篇)

B.1 极对数测量

方法1: 示波器法

1. 手动旋转电机一圈
2. 任意两相之间接示波器 (电机作为发电机)
3. 数正弦波周期数 = 极对数

方法2: 开环法

1. 开环强拖, Theta 增加 2π (电角度一圈)
2. 观察电机机械转了多少角度
3. 极对数 = $2\pi / \text{机械角度}$

例如: Theta 增加 2π , 电机机械转了 $360^\circ/7 \approx 51.4^\circ$, 说明极对数 = 7

B.2 相电阻测量

精密方法 (用电源):

1. 用直流电源给任意两相通电 (例如 1A)
2. 用万用表测这两相之间的电压 V_{meas}
3. $R_{\text{line}} = V_{\text{meas}} / I$
4. $R_{\text{phase}} = R_{\text{line}} / 2$

B.3 相电感测量

LCR 表: 频率选 1kHz 或 10kHz, 测任意两相 → $L_{\text{phase}} = L_{\text{line}} / 2$

无 LCR 表示波器法:

1. 给任意两相通 50% 占空比的 PWM (不转的状态)
2. 串一个已知电阻 R_{sense} (如 0.1Ω)
3. 测电流纹波 ΔI
4. $L = V_{\text{dc}} \times D \times (1-D) \times T_{\text{pwm}} / \Delta I$
D = 0.5 时最简: $L = V_{\text{dc}} \times T_{\text{pwm}} / (4 \times \Delta I)$

附录C：调试命令参考（通过 VOFA+ 发送）

建议在 USART 接收中断中解析以下简易命令：

- help — 打印帮助信息
- stop — 紧急停机
- start — 启动电机
- speed <val> — 设置目标速度 (rpm)
- iq <val> — 设置目标 Iq (A)
- pid <k> <val> — 在线修改 PID 参数
- vq <val> — 设置开环 Vq (V)
- mode <0/1/2> — 切换模式: 0=stop, 1=openloop, 2=closedloop
- param — 打印当前所有参数
- calib — 标定 ADC 零偏
- smo — 打印 SMO 状态 (角度/速度/误差)

附录D：常见故障排查表

故障现象	排查步骤
电机不转	1. 查 EN 引脚是否拉高 2. 查 PWM 是否输出 (示波器) 3. 查 V_q 是否太小 (<1V) 4. 查 FD6288Q 供电 (12V)
电机抖动/异响	1. 查三相 PWM 接线顺序 2. 查死区时间是否配置 3. 查 V_q_target 是否突变
MOSFET / FD6288Q 过热	1. 查死区时间 (太小→直通) 2. 查 PWM 频率 (太高→开关损耗大) 3. 查是否有上下管同时导通
电流波形噪声大	1. 查 ADC 采样时机 (应在PWM中心) 2. 查采样电阻焊接/运放电路 3. 查 DMA 配置是否正确
SMO 角度发散	1. 查电机参数 R/L 是否正确 2. 增大 K_slide 3. 查 V_alpha/V_beta 是否传入正确
闭环切入失败	1. 先开环验证 SMO 角度跟踪 2. 增大切入速度阈值

| | 3. 延长过渡时间 |

| 姿态角漂移 | 1. 查陀螺仪零偏校准 |

| | 2. 增大 Mahony 的 Kp |

| | 3. 查加速度计是否受电机振动影响 |

=====
=====

文档结束
版本: v2.0 (详尽版)
日期: 2026-05-23

=====
=====